

# J.A.R.V.I.S. v2 — Full Phase Planning (v3)

Multi-Agent AI Company | Full Cognitive Architecture | Context Management | Self-Learning

|                      |                     |                         |                      |                  |               |                        |               |                    |
|----------------------|---------------------|-------------------------|----------------------|------------------|---------------|------------------------|---------------|--------------------|
| v1 Ph1-3<br>Planning | v1 Ph4<br>Implement | v1 Ph5-6<br>Test/Deploy | v2 Ph1<br>Foundation | v2 Ph2<br>Agents | v2 Ph3 Vision | v2 Ph4<br>Design/Learn | v2 Ph5 Online | v2 Ph6<br>Security |
|----------------------|---------------------|-------------------------|----------------------|------------------|---------------|------------------------|---------------|--------------------|

## 1. v2 Vision

|                 |                                                                                                                                                                                                                       |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Company type    | Startup builder + Software agency + Research lab + Personal productivity empire                                                                                                                                       |
| Operation       | Background always-on: agents work while you talk, sleep, or are away                                                                                                                                                  |
| Availability    | Local (Windows PC) + Online (cloud, accessible from phone/browser)                                                                                                                                                    |
| Deployment gate | Human review + approval before anything deploys to the real world                                                                                                                                                     |
| AI models       | Mix: Claude/GPT-4o (critical) + Ollama/Mistral local (background, cost control)                                                                                                                                       |
| Memory model    | Full 8-system cognitive architecture: Mem0 + knowledge graph (Neo4j) + episodic store + procedural library + confidence scoring + metacognitive map + nightly consolidation + forgetting curve + spreading activation |
| Learning        | RAG-based — not weight modification. Richer context per call via accumulated memory across all 8 systems. Agents do not become smarter — they become better-informed through accumulated, consolidated knowledge.     |

## 2. Cognitive Architecture — Full Specification

**Critical architecture note — what 'learning' means:** The LLM (Claude, GPT-4o, Mistral) never changes. Its weights are frozen and trained by its provider. What improves over time is the *context injected into every API call* — richer notes, better-organised facts, deeper domain knowledge retrieved from the memory systems below. This pattern is called RAG (Retrieval Augmented Generation). Every major AI product (Perplexity, Cursor, GitHub Copilot, NotebookLM) uses it. The LLM is the cortex — general intelligence, frozen. The memory architecture is the hippocampus — experience, accumulated over time.

J.A.R.V.I.S. implements a cognitive architecture modelled on the human brain. The human brain is not one memory system — it is eight distinct systems operating simultaneously, each with different storage mechanisms, retrieval triggers, and decay rates. This architecture replicates all eight using purpose-built AI libraries and custom components. The result is the closest any software system can come to human-like learning, recall, and knowledge accumulation without modifying the underlying LLM weights.

## 2a. The 8 memory systems

|                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Semantic memory</b><br><i>General world knowledge — facts without personal context</i>     | ChromaDB vector store + NetworkX/Neo4j knowledge graph. Stores facts, concepts, relationships. Retrieved by semantic similarity search. This is the foundation — everything else builds on top of it.<br><b>v2 status:</b> Already planned (v1 core/memory.py + v2 knowledge_graph.py)                                                                                          |
| <b>2. Episodic memory</b><br><i>Personal experiences anchored in time and sequence</i>           | SQLite episodic store. Every agent action is logged with: timestamp, agent_id, task_id, action_type, inputs, outputs, success_score. Answers 'what happened, when, and what was the outcome?' This is what makes agents feel like they have a genuine history.<br><b>v2 status:</b> New — add to v2 Phase 1 (core/episodic_store.py)                                            |
| <b>3. Procedural memory</b><br><i>How to do things — reusable skills and patterns</i>            | Skill library (JSON + Python). After 5+ similar tasks, the consolidation job extracts a reusable pattern and stores it as a named skill. Future tasks load the skill as a prompt prefix. Agents stop re-figuring-out things they have done many times.<br><b>v2 status:</b> New — add to v2 Phase 1 (core/skill_library.py)                                                     |
| <b>4. Working memory</b><br><i>Active conscious focus — the 4–7 items currently in mind</i>      | Context window (200k tokens) + rolling deque(maxlen=20) conversation buffer. Limited, volatile, fast. The context budget governs what fits: system prompt (2k) + memories (3k) + conversation (10k) + task (5k) + tools (8k).<br><b>v2 status:</b> Already planned (v1 core/memory.py deque buffer)                                                                             |
| <b>5. Associative memory</b><br><i>Spreading activation — related concepts light up together</i> | Knowledge graph traversal. When retrieving node X, also retrieve its 1-hop and 2-hop neighbours. 'JWT' activates 'authentication' activates 'bcrypt' activates 'the security bug from last month.' Enables non-obvious connections that pure similarity search cannot make.<br><b>v2 status:</b> New — add to v2 Phase 1 (core/knowledge_graph.py spreading_recall())           |
| <b>6. Prospective memory</b><br><i>Intentions about the future — planned actions</i>             | APScheduler + SQLite task queue. Tasks have a scheduled_at timestamp. 'Check CVE feed every Friday at 9am.' 'Run memory consolidation nightly at 2am.' 'Review Junior Dev output every time a PR is created.' Agents remember to do things without being reminded.<br><b>v2 status:</b> Already planned (v2 core/task_queue.py + APScheduler)                                   |
| <b>7. Confidence memory</b><br><i>Emotional signals — difficulty and success scoring</i>         | Activation scoring system. Every memory has a float confidence_score (0.0–1.0). Score spikes on successful retrieval and use. Score decays daily (Ebbinghaus curve). Low-confidence memories trigger the agent to research before acting. High-confidence memories are acted on directly.<br><b>v2 status:</b> New — add to v2 Phase 1 (metadata field on all ChromaDB entries) |

|                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>8. Metacognitive memory</b><br><i>Knowing what you know</i><br>— competence<br>self-awareness | Per-agent competence map (JSON). Each agent maintains topic coverage scores: {'FastAPI': 0.9, 'cryptography': 0.3, 'React': 0.7}. Score below 0.5 triggers autonomous research before starting the task. Score above 0.8 allows direct execution. Agents know their own limits.<br><b>v2 status:</b> New — add to v2 Phase 2 (agents/base_agent.py competence_map) |
|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 2b. Cognitive mechanisms

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mem0</b>                 | Mem0 (pip install mem0ai) is the unified memory API layer. It wraps ChromaDB, the knowledge graph, and structured memory in a single interface: <code>memory.add(text, agent_id='senior_dev')</code> , <code>memory.search(query, agent_id='senior_dev')</code> , <code>memory.get_all(agent_id='senior_dev')</code> . Mem0 automatically extracts what is worth remembering from conversations without manual [REMEMBER:] tagging — it uses its own extraction LLM call. It is used in production by major AI products and is actively maintained. Mem0 handles semantic + associative memory. Episodic, procedural, confidence, and metacognitive memory are built on top as custom modules. |
| <b>Knowledge graph</b>      | The knowledge graph stores entities and named relationships between them. Node types: Agent, File, Concept, Bug, Project, KnowledgeSource, Skill. Edge types: BUILT, LEARNED, LEARNED_FROM, FOUND, FIXED, DEPENDS_ON, CALLS, BELONGS_TO, USED_IN, REVIEWED, TAUGHT, HAS_BUG. Technology: NetworkX (v1/early v2, in-memory, persisted to JSON) → Neo4j (late v2, full graph database, Cypher query language). Populated automatically: every agent action writes edges. Static analysis (Python ast module) extracts CALLS relationships from code files. Spreading activation retrieves 2-hop neighbours on every query.                                                                       |
| <b>Memory consolidation</b> | Memory consolidation is the most important brain process missing from most AI systems. In the human brain, the hippocampus consolidates memories during sleep — compressing raw episodic experiences into semantic facts stored in the cortex. For Jarvis, a nightly background job runs the same process: it reads today's episodic log, asks the LLM to extract durable facts, stores them in ChromaDB and the knowledge graph, applies the forgetting curve decay to all existing memories, and prunes memories whose activation score has fallen below the threshold. Agents wake up the next day with distilled knowledge rather than a growing pile of raw logs.                         |
| <b>Forgetting curve</b>     | Ebbinghaus (1885) mapped human forgetting mathematically: $R = e^{(-t/S)}$ where $t$ is elapsed time and $S$ is memory strength. Implemented in Jarvis as: every memory has a float activation score starting at 1.0. Score decays by 5% daily. Score spikes by +0.3 on every retrieval (reinforcement). Memories below 0.1 are archived to cold storage (not deleted — can be manually recalled). This means agents naturally retain frequently-used knowledge and let rarely-used knowledge fade — exactly like humans.                                                                                                                                                                      |

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Spreading activation</b> | <p>When the knowledge graph retrieves a node, it also retrieves all nodes within 2 hops. Query 'JWT authentication' → activates JWT node → spreads to: auth.py (USED_IN), bcrypt (RELATED_TO), Bug#7 (FOUND_IN auth.py), Senior_Dev_Agent (BUILT auth.py). All activated nodes are injected into the LLM context. The LLM sees a richer, more connected picture — not just a single matching fact but an entire related cluster. This is what makes Jarvis feel associative rather than merely database-like.</p>                                                                                                                                                                                                                                                                                                               |
| <b>Context management</b>   | <p>Context management is the engineering problem that determines whether the cognitive architecture works in practice. Every LLM API call has a fixed token budget. The build_context() function in core/memory.py assembles the prompt in priority order: (1) system prompt [2,000 tokens], (2) top-3 semantic memory recall [1,500 tokens], (3) spreading activation neighbours [1,500 tokens], (4) episodic summary for current task [1,000 tokens], (5) active project record [1,500 tokens], (6) task context object [5,000 tokens], (7) conversation history deque [10,000 tokens], (8) current user input [1,000 tokens]. Total: ~23,500 tokens — well within the 200k limit with room for tool results. If any section overflows its budget, it is compressed by a summarisation LLM call, never truncated blindly.</p> |

### 3. Full Agent Roster — 14 Agents

**CEO — Jarvis Core | Model: Claude Opus**

**Tools:** All tools, voice, vision, delegation, goal decomposition  
**Self-learning:** Learns from project outcomes. Refines delegation based on agent performance history in episodic store.

**Product Manager | Model: Claude Sonnet / GPT-4o**

**Tools:** Requirements, roadmap, tickets (GitHub Issues), web research  
**Self-learning:** Reads PM frameworks (RICE, OKRs, Jobs-to-be-Done) via YouTube transcripts + blogs. Stores in procedural memory.

**Project Manager | Model: Claude Haiku / Mistral (local)**

**Tools:** Task queue, deadlines, progress reports, Gantt charts  
**Self-learning:** Learns from sprint retrospectives in episodic store. Improves time estimates from historical task data.

**Senior Developer | Model: Claude Opus / GPT-4o**

**Tools:** Full-stack code, architecture, GitHub push/PR, Docker  
**Self-learning:** Reads official docs + GitHub trending repos. Mentors Junior Dev via structured feedback stored in knowledge graph.

**Junior Developer | Model: Claude Sonnet / Gemini Flash**

**Tools:** Implement tasks from Senior Dev, unit tests, docs  
**Self-learning:** Receives structured feedback from Senior Dev (TAUGHT edges in knowledge graph). Reads assigned docs before each task.

**DevOps / Infrastructure | Model: Claude Haiku / Mistral (local)**

**Tools:** Docker, GitHub Actions CI/CD, server provisioning, deployment  
**Self-learning:** Reads DevOps blogs + AWS docs. Stores successful deployment patterns in procedural memory.

**Cybersecurity Analyst | Model: Claude Sonnet / GPT-4o**

**Tools:** Static analysis, pip-audit, OWASP checklist, threat modelling  
**Self-learning:** Reads CVE feeds daily (prospective memory scheduler). Maintains vulnerability knowledge base in ChromaDB.

**Ethical Hacker (guarded) | Model: Claude Opus + whitelist**

**Tools:** Pen testing on authorized\_targets only — hardcoded whitelist  
**Self-learning:** Studies OWASP Top 10, CTF writeups. ALL actions require CEO + user double confirmation.

**UI/UX Designer | Model: Claude Vision + DALL-E 3 / Flux**

**Tools:** Wireframes, UI components, image assets, design system

**Self-learning:** Watches UI/UX YouTube tutorials (transcript learning). Studies Dribbble trends. Builds design knowledge base.

### Brand & Content Designer | Model: Claude + DALL-E 3 / Flux

**Tools:** Logos, marketing copy, social assets, PowerPoint decks

**Self-learning:** Learns from brand case studies and advertising research papers stored in ChromaDB.

### Research Analyst | Model: Claude Opus / Gemini Pro

**Tools:** Deep web search, arXiv papers, YouTube transcripts, PDF analysis

**Self-learning:** Builds continuously growing knowledge base. Identifies knowledge gaps for nightly research scheduling.

### Data Scientist / ML | Model: Claude + Python kernel

**Tools:** Data analysis (pandas), charts (plotly), ML (scikit-learn)

**Self-learning:** Reads Kaggle notebooks + ML papers. Stores reusable analysis patterns in procedural memory.

### QA Engineer | Model: Claude Sonnet / Gemini Flash

**Tools:** pytest writer, integration tests, bug reports, regression detection

**Self-learning:** Studies testing best practices. Improves test patterns based on bugs found in episodic store.

### Code Reviewer | Model: Claude Opus

**Tools:** PR reviews, style enforcement (black/flake8/eslint), profiler

**Self-learning:** Builds personal code quality ruleset from reviewing projects. Stores patterns in knowledge graph.

## 4. v2 Functional Requirements

| ID       | Category      | Requirement                                                                                     |
|----------|---------------|-------------------------------------------------------------------------------------------------|
| FR-V2-01 | Background    | Agents run as asyncio background tasks while voice loop remains responsive                      |
| FR-V2-02 | Delegation    | CEO agent decomposes goals into tasks and assigns to agents without further user input          |
| FR-V2-03 | Status        | Agents post status to task queue — user can query 'what is the team working on?' at any time    |
| FR-V2-04 | Non-interrupt | Background agents do not interrupt voice sessions unless critical decision is required          |
| FR-V2-05 | Persistence   | All in-progress work persisted to SQLite — agents resume after restart                          |
| FR-V2-06 | Vision        | CEO agent takes screenshots, analyses with vision LLM, acts via pyautogui based on what it sees |
| FR-V2-07 | Research      | Agents browse web, read PDFs, fetch YouTube transcripts autonomously for assigned topics        |
| FR-V2-08 | Local API     | FastAPI server runs on localhost:8000 as backend for dashboard and cloud sync                   |
| FR-V2-09 | Online        | FastAPI deployable to Railway/Render — dashboard accessible from phone and browser              |
| FR-V2-10 | Sync          | SQLite (local) syncs to PostgreSQL (cloud) every 5 minutes                                      |
| FR-V2-11 | Auth          | All API endpoints require JWT authentication — no public access                                 |
| FR-V2-12 | Task queue    | Every task has: ID, agent, status (pending/active/blocked/done), priority, input, output        |
| FR-V2-13 | Delegation v2 | Agents spawn sub-tasks and assign to other agents (e.g. Dev asks Research for docs)             |
| FR-V2-14 | Mentoring     | Senior Dev reviews Junior Dev output and stores structured feedback in knowledge graph          |
| FR-V2-15 | Episodic      | All agent actions logged to episodic store with timestamp, agent_id, success_score              |
| FR-V2-16 | Consolidation | Nightly consolidation: episodic → semantic facts, forgetting curve decay, low-score archiving   |
| FR-V2-17 | Competence    | Each agent maintains JSON competence map. Score <0.5 triggers research before task execution    |

|          |           |                                                                                              |
|----------|-----------|----------------------------------------------------------------------------------------------|
| FR-V2-18 | Graph v2  | Knowledge graph migrated to Neo4j. Cypher queries for complex relationship traversal         |
| FR-V2-19 | Dashboard | React dashboard: agent status, Kanban board, live log stream (WebSocket), output viewer      |
| FR-V2-20 | Learning  | Designer agent learns UI/UX from YouTube transcripts. Dev agent reads official documentation |

## 5. v2 Non-Functional Requirements

| ID        | Requirement                                                                                |
|-----------|--------------------------------------------------------------------------------------------|
| NFR-V2-01 | Background agents consume <40% CPU when autonomous — async rate limiting + sleep intervals |
| NFR-V2-02 | Dashboard API responses <500ms (excluding async LLM calls)                                 |
| NFR-V2-03 | All inter-agent messages persisted — no message lost on restart (SQLite WAL mode)          |
| NFR-V2-04 | JWT tokens expire after 24h. Refresh tokens in httpOnly cookies only                       |
| NFR-V2-05 | Ethical hacker requires double confirmation: CEO approval + explicit user voice/click      |
| NFR-V2-06 | All actions modifying filesystem/network/UI logged with timestamp + agent + action details |
| NFR-V2-07 | 10 concurrent agents without degradation on 16GB RAM machine                               |
| NFR-V2-08 | Self-learning tasks run at lowest priority — never block voice responses                   |
| NFR-V2-09 | Dashboard mobile-responsive at 375px minimum viewport                                      |
| NFR-V2-10 | Cloud sync uses HTTPS end-to-end — no agent outputs transmitted in plaintext               |

## 6. v2 Implementation Roadmap

| Phase   | Duration  | What gets built                                                                                                                                                                                                                | Deliverable                                                                     |
|---------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| v2 Ph 1 | 2–3 weeks | asyncio background worker, SQLite task queue, CEO agent v2, full cognitive architecture upgrade (Mem0, NetworkX knowledge graph, episodic store, consolidation, forgetting curve, spreading activation), context budget module | 2 agents run concurrently. Nightly consolidation runs. Memory feels persistent. |

|         |           |                                                                                                                                                                                           |                                                                                 |
|---------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| v2 Ph 2 | 3–4 weeks | PM, Senior Dev, Junior Dev, QA, Research agents. Agent-to-agent delegation. Senior Dev teaches Junior Dev. Research agent autonomous web/PDF/YouTube learning. Per-agent competence maps. | Give Jarvis a coding goal — the dev team builds it in the background.           |
| v2 Ph 3 | 2–3 weeks | Screen vision (pyautogui screenshot + Claude vision API), pyautogui action execution from vision output, screen error reading → relay to Dev agent                                        | Jarvis sees your screen, operates your PC autonomously.                         |
| v2 Ph 4 | 3–4 weeks | Designer agent (DALL-E 3 / Flux), Brand agent, YouTube transcript learning engine, autonomous research scheduler, design knowledge base from tutorials                                    | Designer creates UI assets, learns from tutorials, improves with every project. |
| v2 Ph 5 | 3–4 weeks | FastAPI server, JWT auth, React dashboard (Kanban + agent status + WebSocket log stream + output viewer), SQLite→PostgreSQL sync, cloud deploy                                            | Full dashboard accessible from phone. Jarvis available online.                  |
| v2 Ph 6 | 2–3 weeks | Cybersecurity agent, ethical hacker (whitelist guarded), DevOps agent, Neo4j knowledge graph migration, full audit logging, 10-agent concurrent performance hardening                     | Complete 14-agent AI company. Stable v2.0.0 release.                            |

## 7. Full SDLC — v1 + v2 Combined

| Phase      | Status | Description                                                           |
|------------|--------|-----------------------------------------------------------------------|
| v1 Phase 1 | DONE   | Planning — Project Charter (v3, cognitive arch added)                 |
| v1 Phase 2 | DONE   | Requirements — SRS (v3, 10 cognitive FRs + 5 cognitive NFRs added)    |
| v1 Phase 3 | DONE   | Design — Modular Monolith + full cognitive architecture (v3)          |
| v1 Phase 4 | ACTIVE | Implementation — 10 steps including full cognitive memory system (v4) |
| v1 Phase 5 | NEXT   | Testing — unit + integration + UAT + memory system tests              |
| v1 Phase 6 | NEXT   | Deployment — setup.bat, user guide, CHANGELOG.md, v1.0.0 tag          |
| v2 Phase 1 | FUTURE | Foundation — async worker, task queue, full cognitive arch upgrade    |
| v2 Phase 2 | FUTURE | Core agent team — PM, Dev (Sr+Jr), QA, Research                       |
| v2 Phase 3 | FUTURE | Vision + autonomous screen operation                                  |
| v2 Phase 4 | FUTURE | Design layer + self-learning engine (YouTube transcripts)             |

|            |        |                                                          |
|------------|--------|----------------------------------------------------------|
| v2 Phase 5 | FUTURE | Online layer — FastAPI, React dashboard, cloud deploy    |
| v2 Phase 6 | FUTURE | Security layer, DevOps agent, Neo4j migration, hardening |

**Sequencing note:** v2 does NOT begin until v1 Phase 5 (Testing) is complete and v1 Phase 6 produces a stable v1.0.0 release. The multi-agent system requires the v1 cognitive core to be solid and fully tested. The v1 cognitive architecture (Mem0, knowledge graph, episodic store, consolidation) is the foundation on which all 14 v2 agents are built.